

ARAMIREZ-AI

Arquitectura del Sistema

Plataforma Unificada — Componentes y Flujos

Organización	Nombre de la Organización
Preparado por	Software Architect
Fecha	28 Jun 2026
Clasificación	Interno

1. Visión General de la Arquitectura

La Plataforma Unificada está compuesta por 5 microservicios orquestados a través de un API Gateway (Fastify). La comunicación entre servicios es síncrona vía HTTP/REST para consultas en línea y asíncrona vía RabbitMQ para comandos y eventos. Cada servicio despliega en contenedores Docker orquestados por Kubernetes.

Componente	Rol	Tecnología	Réplicas
API Gateway	Proxy inverso, rate limiting, autenticación	Fastify + Envoy	3
Auth Service	Autenticación y autorización (OAuth 2.0)	Node.js + JWT	2
User Service	CRUD de usuarios y perfiles	Node.js + PostgreSQL 16	3
Notification Service	Envío de notificaciones (email, push, SMS)	Node.js + RabbitMQ	2
Dashboard Service	Métricas agregadas en tiempo real	Node.js + Redis	2
Frontend	Interfaz de usuario SPA	React 19 + Vite	2

- Flujo de consulta: Cliente → API Gateway → Auth (JWT) → User Service → PostgreSQL
- Flujo de eventos: User Service publica → RabbitMQ → Notification Service consume
- Caché distribuido: Redis para sesiones y métricas en tiempo real
- Monitoreo: Prometheus + Grafana en cluster dedicado de observabilidad
- CI/CD: GitHub Actions → Docker Registry → ArgoCD → Kubernetes

2. Decisiones Técnicas y Recomendaciones

Decisión Arquitectónica Clave

Se eligió RabbitMQ sobre Kafka debido a que el volumen de eventos (< 10,000 msg/día) no justifica la sobrecarga operativa de Kafka. RabbitMQ ofrece enrutamiento flexible, colas dead-letter y management UI integrado que facilitan la operación diaria.

Implementar Circuit Breaker en API Gateway

Problema: Actualmente no existe protección contra fallos en cascada: si un servicio upstream falla, el API Gateway sigue enviando tráfico, saturando conexiones y afectando a servicios saludables

Recomendación: Implementar el patrón Circuit Breaker en el API Gateway usando la biblioteca opossum para Node.js, con umbral de 5 fallos en 30 segundos

Acciones sugeridas:

- Integrar opossum en las rutas del API Gateway que consumen microservicios
- Configurar umbral: 5 fallos consecutivos en ventana de 30 segundos
- Definir respuesta fallback (HTTP 503 + mensaje amigable)
- Monitorear estado del circuit breaker en Prometheus
- Realizar prueba de caos para validar el comportamiento



Arquitectura documentada bajo el estándar C4 (Contexto, Contenedores, Componentes, Código). Diagrama disponible en /assets/images/architecture-diagram.png. Próxima revisión programada: Q4 2026.